

CLI Agent Local Checker Profile v0.1

Deterministic preflight, package hygiene, schema validation, semantic guardrails, and conformance-readiness checks for C-Governed CLI Agent Mesh operations

Status	Draft normative profile v0.1
Date	2026-05-17
Package	C-Governed CLI Agent Mesh
Layer	c = a + b / SER / L4 / CLI Worker Mesh / Local Validation / Package Hygiene / Conformance / Witness
Document class	local checker profile / implementation-readiness artifact / fail-closed validation companion
Assertion class	C-A10 package-control artifact; C-A7 where hash, canonicalization, witness-linking, or verification claims are made
Primary subject	persistent c entities using local, cloud, or hybrid CLI agents as bounded executable workers
Primary boundary	the local checker is a deterministic inspection layer. It may validate structure, policy alignment, gate readiness, stale claims, fixture safety, and evidence references. It must not become will, memory, judge, release authority, incident authority, legal authority, or a self-authorizing executor.

Kotov Ivan

Bruxelles, 2026

Contents

0. Executive definition	5
1. Purpose	5
2. Non-goals	6
3. Corpus bridge set	7
3.1 Explicit bridge: $c = a + b$	7
3.2 Quiet bridge I: information theory	7
3.3 Quiet bridge II: cybernetics	7
3.4 Earth paragraph	7
4. Core rule	7
5. Checker authority limits	8
5.1 Allowed checker actions	8
5.2 Forbidden checker actions	8
6. Checker modes	9
6.1 Mode table	9
6.2 Default mode	9
7. Result vocabulary	10
7.1 Finding status	10
7.2 Overall result	10
7.3 Fail-closed rule	10
8. Mandatory checker domains	10
8.1 Package inventory domain	10
8.2 Status hygiene domain	11
8.3 Schema domain	11
8.4 Semantic guard domain	11
8.5 AB/gate domain	12

8.6 Registry domain	12
8.7 Task contract / permission domain	13
8.8 Sandbox/worktree domain	13
8.9 Witness domain	13
8.10 Raw evidence sidecar domain	14
8.11 Memory gate domain	14
8.12 Quorum / role separation domain	14
8.13 Fixture domain	14
8.14 Public/restricted release domain	15
8.15 Release/public surface domain	15
8.16 Conformance domain	16
9. Object model	17
9.1 CLI_AGENT_LOCAL_CHECKER_PROFILE	17
9.2 CLI_AGENT_CHECKER_RUN	18
9.3 CLI_AGENT_CHECKER_FINDING	18
9.4 CLI_AGENT_CHECKER_RESULT	19
10. Claim-control mapping	19
11. Checker precedence	19
12. Local checker and Codex handoff	20
13. Local checker and SYNAPS-style handoff	20
14. Required finding classes	21
15. Required blocker classes	21
16. Safe report content	22
17. Implementation target layout	22
18. Minimal CLI shape	23
19. A/B behavior	23
19.1 Slot A	23

19.2 Slot B	23
20. Conformance tests for the checker	24
21. Relation to conformance	24
22. Review and challenge	24
23. Open issues	25
24. Release-readiness impact	25
25. Final invariant	25

0. Executive definition

CLI Agent Local Checker Profile defines the local, deterministic validation layer used before a CLI/cloud agent task, package release, schema extraction, conformance run, memory promotion, incident repair, or public handoff is treated as safe enough to proceed.

The local checker answers:

- Are the required files present?
- Are canonical names consistent?
- Are stale missing-file claims still present?
- Are duplicate variants unresolved?
- Are schemas present and structurally valid?
- Are examples structurally valid?
- Are semantic red lines violated?
- Are AB/apply gates coherent?
- Are registry entries eligible?
- Are task contracts and permission grants aligned?
- Are sandbox/worktree boundaries declared?
- Are witness references present where required?
- Are raw evidence sidecars separate from witness and memory?
- Are fixtures synthetic and safe?
- Are public/restricted surfaces separated?
- Are conformance claims supported by actual results?

Compact formula:

- The checker measures.
- It does not decide sovereignty.
- It blocks ambiguity.
- It does not invent authority.

A local checker is not an LLM reviewer. A local checker is not a Judge. A local checker is not the human anchor. A local checker is not c. A local checker is not release authority.

It is the electrical tester before energizing the panel.

1. Purpose

The C-Governed CLI Agent Mesh package already contains task contracts, permission models, handshakes, sandbox/worktree rules, witness events, memory gates, rollback/freeze semantics, quorum review, executor/reviewer separation, defensive emulation boundaries, incident response, secrets/cloud-data policy, conformance fixtures, AB/gate rules, agent registry rules, and raw evidence sidecar handling.

That stack is useful only if implementation workers do not accidentally bypass it.

The local checker exists to prevent:

1. stale package status;
2. duplicate canonical files;
3. schema drift;
4. policy text and executable checks drifting apart;
5. AB/apply inversions;
6. hidden self-approval;

7. conformance claims without evidence;
8. fixture abuse leakage;
9. public/restricted surface leakage;
10. witness/evidence/memory confusion;
11. registry entries being treated as permission grants;
12. task contracts being treated as active permission grants;
13. raw evidence being written into witness or memory;
14. release claims stronger than available artifacts;
15. Codex or another CLI agent using the weakest wording in the package.

The local checker is therefore not cosmetic. It is the first implementation spine.

2. Non-goals

This profile does not define or permit:

1. autonomous execution;
2. autonomous repair;
3. autonomous release;
4. autonomous publication;
5. autonomous memory promotion;
6. autonomous incident authority;
7. autonomous legal/security reporting;
8. offensive cyber operations;
9. hack-back;
10. live external exploitation;
11. malware behavior;
12. credential theft;
13. covert persistence;
14. evasion;
15. unauthorized scanning;
16. destructive action outside authorized scope;
17. deletion of evidence as a substitute for review;
18. agent self-certification;
19. use of LLM prose as a validator result;
20. treating a clean checker run as proof of overall safety.

A local checker may produce evidence for review.

It does not replace review.

3. Corpus bridge set

3.1 Explicit bridge: $c = a + b$

In $c = a + b$, the local checker belongs to b : the technological substrate of procedures, schemas, validators, hashes, manifests, fixture checks, status checks, and fail-closed gates.

The checker does not become c .

It constrains worker agents so that parts of b do not silently become will, memory, judge, or release authority.

3.2 Quiet bridge I: information theory

The checker reduces ambiguity. It turns prose requirements into explicit pass/warn/fail/block findings. It cannot decide meaning by itself, but it can prevent structurally invalid or semantically dangerous objects from being smuggled through fluent text.

3.3 Quiet bridge II: cybernetics

A control loop needs sensors. The local checker is a sensor and interlock: it detects drift, missing gates, stale claims, malformed objects, and unsafe transitions before the mesh applies them. It is negative feedback, not command authority.

3.4 Earth paragraph

Before energizing a renovated electrical panel, you do not ask the apprentice whether the wiring “feels correct.” You test continuity, grounding, labels, breaker mapping, insulation, and load paths. The tester does not own the building. It simply refuses to let bad wiring become a fire. This profile gives the CLI-agent mesh that tester.

4. Core rule

The local checker must be stricter than prose interpretation and weaker than final authority.

- Prose may explain.
- Schemas may shape.
- The checker may block.
- Review may interpret.
- c may integrate.
- The human anchor handles high-risk consequence.

The checker must never silently upgrade a status.

It may downgrade, block, quarantine, or require review.

5. Checker authority limits

5.1 Allowed checker actions

The local checker may:

1. read package files;
2. read extracted schemas;
3. read manifests;
4. read fixture definitions;
5. read task contracts;
6. read registry records;
7. read permission grants;
8. read checker policy files;
9. read witness references;
10. read evidence-packet metadata;
11. read sidecar metadata without exposing raw evidence;
12. compute hashes;
13. validate JSON/YAML/Markdown structure;
14. detect duplicate canonical names;
15. detect missing required files;
16. detect stale status claims;
17. detect forbidden claims;
18. detect red-line terms and unsafe fixture patterns;
19. detect gate inconsistencies;
20. write checker reports when explicitly allowed.

5.2 Forbidden checker actions

The local checker must not:

1. modify protocol documents during validation;
2. modify task contracts during validation;
3. modify registry records during validation;
4. modify permission grants during validation;
5. write to c memory;
6. promote memory;
7. clear quarantine;
8. unfreeze paths;

9. approve release;
10. publish public artifacts;
11. execute repair commands;
12. fetch live external targets;
13. upload private material to cloud contexts;
14. expose raw evidence;
15. decode or print secrets;
16. convert a warning into approval;
17. convert a missing file into “not applicable” without declared policy;
18. treat an LLM answer as validation authority.

6. Checker modes

6.1 Mode table

Mode	Name	Purpose	Writes allowed	Release impact
LC-0	Inventory	list files, names, hashes, duplicates	report only	no claim
LC-1	Package hygiene	check canonical files, stale status, reading order, release notes	report only	release-preflight
LC-2	Schema structural	validate JSON/YAML objects against extracted schemas	report only	implementation-preflight
LC-3	Semantic guard	enforce red lines, gate rules, role separation, AB semantics	report only	implementation-preflight
LC-4	Fixture safety	check synthetic fixtures and expected results	report only	conformance-preflight
LC-5	Release surface	check public/restricted split, README, index, manifest, SHA plan	report only	release-preflight
LC-6	Runtime task preflight	check one planned task contract against registry, permission, sandbox, AB, witness	report only unless runner explicitly calls it as a gate	task-preflight
LC-7	Evidence / incident preflight	check evidence packet references, sidecar metadata, quarantine clear prerequisites	report only unless incident gate explicitly calls it	incident-preflight
LC-8	Conformance run collector	collect fixture run results and evidence packet references	report only	conformance-readiness

6.2 Default mode

Default mode is LC-0 or LC-1 with no mutation.

A checker must not enter LC-6, LC-7, or LC-8 unless the requested mode is explicit.

7. Result vocabulary

7.1 Finding status

Status	Meaning	Default consequence
PASS	requirement satisfied	may continue to next check
INFO	informational observation	no block
WARN	weakness or maturity gap	review before strong claim
FAIL	requirement violated	block affected claim or action
BLOCK	safety/release/conformance blocker	fail closed
UNKNOWN	checker cannot determine state	fail closed for material gates
NOT_APPLICABLE	explicitly out of scope	allowed only with reason
DEFERRED	known future work	blocks strong claim if required for that claim

7.2 Overall result

PASS = all required checks passed for requested gate
 PASS_WARN = no blockers, but warnings remain
 FAIL = at least one required check failed
 BLOCK = at least one fail-closed blocker present
 UNKNOWN = insufficient data for requested gate

7.3 Fail-closed rule

For any material gate:

UNKNOWN == BLOCK

Material gates include:

release-complete
 implementation-ready
 conformance-supported
 public-safe
 memory-promotion
 quarantine-clear
 incident-repair-apply
 task-execute
 runner-enable
 persistent-daemon-enable

8. Mandatory checker domains

8.1 Package inventory domain

The checker must verify:

1. every canonical file listed in the package index exists;
2. no required file is missing;
3. no duplicate canonical variant exists unless explicitly marked as superseded;

4. no filename variant differs only by case, spacing, or suffix such as (1);
5. every newly created file appears in package-control files where required;
6. every restricted file is tagged with a distribution class;
7. every package-control file has date, status, package, document class, and boundary metadata;
8. every accepted artifact contains at least one explicit bridge, two quiet/hidden bridges, and an earth paragraph unless classified as pure machine object.

8.2 Status hygiene domain

The checker must verify:

1. release notes do not claim missing files that now exist;
2. open issues distinguish missing file from file exists but not implemented;
3. contradiction register issue statuses reflect actual file inventory;
4. package status does not overclaim readiness;
5. release-complete status is not claimed before release gate passes;
6. implementation-ready status is not claimed before schema/object/validator/fixture prerequisites are satisfied;
7. conformance-supported status is not claimed before actual runs and evidence packets exist;
8. public-safe status is not claimed before redaction split is checked.

8.3 Schema domain

The checker must verify:

1. schemas/ exists when implementation-ready is claimed;
2. every machine-facing object has a schema or explicit deferral;
3. SCHEMA_INDEX.json exists when implementation-ready is claimed;
4. schema \$id values are stable and versioned;
5. schema versions match Markdown canonical schema versions;
6. examples validate structurally;
7. schema validity is not treated as semantic safety;
8. natural-language placeholders are not present in machine schema examples unless explicitly marked as prose-only.

8.4 Semantic guard domain

The checker must block any object, fixture, task, example, or release note that attempts to authorize:

```
hack-back  
live external counter-operation  
malware behavior
```

credential theft
token extraction
covert persistence
evasion
unauthorized scanning
unauthorized exploitation
autonomous retaliation
secret exfiltration
agent self-approval
direct memory write by agent
identity/core mutation by agent
witness tampering
raw evidence publication
legal privileged disclosure without legal review

8.5 AB/gate domain

The checker must verify:

1. A means observe/dry-run/no durable mutation in global CGAM semantics;
2. B means controlled apply only after required gates;
3. B alone is never authority;
4. local AB inversions are declared;
5. unknown AB semantics block material apply;
6. confirm phrase is present when required;
7. confirm phrase does not replace task contract;
8. apply flag does not replace permission grant;
9. runner gate is distinct from ordinary apply;
10. persistent gate is distinct from runner gate;
11. dry-run writes no durable project state except an explicitly allowed dry-run report;
12. memory=off and auto_ingest=false remain defaults for handoff, transfer, and ledger layers unless a memory gate explicitly overrides.

8.6 Registry domain

The checker must verify:

1. agent has a registry record before task eligibility;
2. registry record has agent_id, provider/runtime class, trust level, auto-connect ceiling, capability snapshot, data boundary, revocation path, and status;
3. registration is not treated as permission;
4. capability is not treated as authority;
5. expired/suspended/quarantined/revoked agents cannot receive material tasks;
6. cloud/hybrid agents have cloud data boundary classification;
7. registry hash/provenance is present where required;
8. USB/node capability exports are treated as snapshots, not trust grants.

8.7 Task contract / permission domain

The checker must verify:

1. every material task has a task contract;
2. requested scope and active permission grant are separated;
3. task contract does not exceed agent registry ceiling;
4. permission grant does not exceed task contract;
5. denied paths and denied data classes are explicit;
6. network policy is explicit;
7. budget is explicit;
8. output requirements are explicit;
9. reviewer requirement is explicit;
10. failure behavior is explicit;
11. invalid contracts resolve to deny/quarantine rather than fallback execution.

8.8 Sandbox/worktree domain

The checker must verify:

1. material writes require sandbox/worktree/container or equivalent isolation;
2. protected branches are not direct write targets;
3. denied paths include memory, identity, witness, secret, legal, release, and continuity surfaces where applicable;
4. dirty-state behavior is declared;
5. rollback plan exists for material changes;
6. diff review path exists;
7. incident repair happens after evidence preservation;
8. integration gate exists before merge/apply.

8.9 Witness domain

The checker must verify:

1. witness-required transitions have witness references;
2. witness events contain minimal boundary records, not raw dumps;
3. witness events do not contain secrets;
4. witness events do not contain private memory by default;
5. witness references link to task, permission, sandbox, registry, and evidence references where required;
6. missing witness on privileged transition blocks the affected claim/action;

7. witness event is not treated as proof of truth or final authority.

8.10 Raw evidence sidecar domain

The checker must verify:

1. raw evidence is not stored inside witness events;
2. raw evidence is not written to memory;
3. raw evidence is not public by default;
4. sidecar metadata exists where raw evidence is referenced;
5. sidecar metadata includes classification, custody, hash, retention, access boundary, and disclosure state;
6. evidence packet references raw evidence by hash/path/ref, not by unredacted content;
7. L4W envelope/selective disclosure does not imply full public disclosure;
8. raw evidence absence blocks claims that require it.

8.11 Memory gate domain

The checker must verify:

1. CLI-agent output is not promoted directly into c memory;
2. memory proposal exists before memory promotion;
3. source, task, witness, and uncertainty fields are present;
4. cloud-generated outputs have cloud boundary markers;
5. incident-derived outputs remain restricted unless reviewed;
6. immunity update is not retaliation;
7. memory rollback/correction is distinct from file rollback;
8. rejected/quarantined outputs do not become experience by accident.

8.12 Quorum / role separation domain

The checker must verify:

1. executor is not sole reviewer;
2. same agent does not approve its own material changes;
3. tester/reviewer/auditor roles are declared where required;
4. same-source consensus risk is marked;
5. quorum output is evidence, not sovereignty;
6. disagreement handling is explicit;
7. high-risk decisions route to c and/or human anchor as required.

8.13 Fixture domain

The checker must verify:

1. fixtures are synthetic;
2. fake secrets are clearly marked fake;
3. no real secrets are present;
4. no live targets are present;
5. no exploit recipes are present;
6. no deployable malware behavior is present;
7. no credential theft logic is present;
8. no evasion procedure is present;
9. no retaliation workflow is present;
10. expected result is declared;
11. fixture risk class is declared;
12. fixture evidence packet expectation is declared where required.

8.14 Public/restricted release domain

The checker must verify:

1. public/restricted/internal distribution classes are assigned;
2. restricted files are not included in public release without redaction;
3. public examples are synthetic;
4. real secrets, real incident traces, private memory, legal material, canary values, provider/account identifiers, local infrastructure details, and defensive garage contents are excluded from public surface;
5. red-line wording remains visible and not softened;
6. README states non-goals and prohibited uses;
7. release notes do not overclaim implementation/conformance/public-safety status.

8.15 Release/public surface domain

The checker must verify:

1. default branch visibility is correct;
2. README entry point is present;
3. package index is present;
4. reading order is present;
5. release notes are present;
6. open issues are present;
7. contradiction register is present;
8. public redaction profile is present;

9. manifest is present before release-complete claim;
10. PDF/derived artifacts are generated only after source freeze;
11. SHA256 manifests are generated only after final artifacts;
12. tag/release is not created before gate pass;
13. Zenodo/site/public pages are not updated from stale package metadata.

8.16 Conformance domain

The checker must verify:

1. conformance matrix exists;
2. safe fixture pack exists;
3. fixture manifest exists when conformance-supported is claimed;
4. schemas exist when conformance-supported is claimed;
5. validator/runner exists when conformance-supported is claimed;
6. actual test runs exist when conformance-supported is claimed;
7. evidence packets exist for actual test runs;
8. red-line drills exist where claimed;
9. provider-specific tests are either present or explicitly deferred;
10. conformance result is scoped and not generalized beyond tested claims.

9. Object model

9.1 CLI_AGENT_LOCAL_CHECKER_PROFILE

```
schema: cli-agent-local-checker-0.1
profile_id: cgam-local-checker-default-v0-1
created_at: "2026-05-17T00:00:00Z"
package: C-Governed CLI Agent Mesh
checker_modes:
  - LC-0
  - LC-1
  - LC-2
  - LC-3
  - LC-4
  - LC-5
  - LC-6
  - LC-7
  - LC-8
default_mode: LC-1
mutation_policy:
  default: report_only
  may_write_reports: true
  may_modify_package_files: false
  may_modify_memory: false
  may_clear_quarantine: false
  may_publish: false
fail_closed:
  material_unknown_blocks: true
  red_line_blocks: true
  missing_required_artifact_blocks: true
  stale_status_blocks_strong_claim: true
required_domains:
  - package_inventory
  - status_hygiene
  - schemas
  - semantic_guard
  - ab_gate
  - registry
  - task_contract_permission
  - sandbox_worktree
  - witness
  - raw_evidence_sidecar
  - memory_gate
  - quorum_role_separation
  - fixtures
  - public_restricted_release
  - release_public_surface
  - conformance
```

9.2 CLI_AGENT_CHECKER_RUN

```
schema: cli-agent-checker-run-0.1
run_id: checker-run-2026-05-17T000000Z
checker_profile_id: cgam-local-checker-default-v0-1
requested_by: human_anchor_or_c
mode: LC-1
scope:
  package_root: ./docs/c-governed-cli-agent-mesh
  include_patterns:
    - "*.md"
    - "schemas/*.schema.json"
    - "fixtures/**/*.json"
    - "manifests/**/*.json"
  exclude_patterns:
    - "restricted/raw/**/*.json"
    - ".git/**/*"
inputs:
  package_manifest: PACKAGE_MANIFEST.json
  schema_index: SCHEMA_INDEX.json
  registry: AGENT_REGISTRY.json
  fixture_manifest: FIXTURE_MANIFEST.json
policy:
  material_unknown_blocks: true
  red_line_blocks: true
  report_only: true
output:
  report_json: reports/local_checker/checker-run-2026-05-17T000000Z.json
  report_md: reports/local_checker/checker-run-2026-05-17T000000Z.md
  sha256: ""
```

9.3 CLI_AGENT_CHECKER_FINDING

```
schema: cli-agent-checker-finding-0.1
finding_id: LC-F-0001
domain: status_hygiene
severity: BLOCK
status: FAIL
location:
  file: CLI_Agent_RELEASE_NOTES_v0_1.md
  section: "8. Open release blockers"
problem: stale_missing_file_claim
diagnosis: file exists but release notes still list it as missing
required_action: update release notes and open-issues status
blocks:
  - release-complete
  - implementation-ready
evidence_refs:
  - kind: file_inventory
    ref: PACKAGE_MANIFEST.json#files
```

9.4 CLI_AGENT_CHECKER_RESULT

```

schema: cli-agent-checker-result-0.1
run_id: checker-run-2026-05-17T000000Z
overall: BLOCK
summary:
  pass: 42
  info: 8
  warn: 6
  fail: 3
  block: 2
  unknown: 1
blocked_claims:
  - release-complete
  - implementation-ready
  - conformance-supported
allowed_claims:
  - draft-architecture
  - release-completion-in-progress
  - implementation-readiness-preparation
next_required_actions:
  - update_release_notes
  - update_open_issues
  - extract_json_schemas
  - create_schema_index
  - run_fixture_validation

```

10. Claim-control mapping

Desired claim	Checker minimum	Required result
draft-architecture	LC-1	PASS_WARN allowed
release-completion-in-progress	LC-1 + LC-5	PASS_WARN allowed
release-complete	LC-1 + LC-5	PASS, no blockers
implementation-readiness-preparation	LC-1 + LC-2 + LC-3	PASS_WARN allowed
implementation-ready	LC-1 + LC-2 + LC-3 + LC-4 + LC-6	PASS, no required unknowns
public-safe	LC-5 + public/restricted domain	PASS, redaction blockers zero
conformance-prepared	LC-2 + LC-3 + LC-4	PASS_WARN allowed
conformance-supported	LC-2 + LC-3 + LC-4 + LC-8	PASS, actual runs and evidence packets present
task-executable	LC-6	PASS, no red-line or unknown material gate
quarantine-clear-ready	LC-7	PASS, valid evidence packet and sidecar references

11. Checker precedence

The checker must apply the following precedence if documents disagree:

```

red-line prohibition
  > legal / safety boundary
  > human-anchor high-risk gate
  > c-governance boundary
  > AB/gate semantics
  > registry eligibility

```

- > task contract
- > permission grant
- > sandbox/worktree boundary
- > witness requirement
- > memory gate
- > release/public surface
- > conformance claim
- > ordinary documentation preference

No lower-level document may relax a higher-level prohibition.

No checker configuration may convert a prohibited action into an allowed action.

12. Local checker and Codex handoff

Before Codex receives an implementation task, the local checker should produce a handoff summary that contains only:

```
checker run id
mode
allowed scope
blocked claims
required files
safe input pointers
hashes
no raw evidence
memory=off
auto_ingest=false
```

Codex must not receive:

```
raw secrets
raw private memory
raw incident evidence
raw legal material
unredacted defensive garage contents
live target details
provider account details
private registry contents beyond scoped synthetic/authorized records
```

The checker report may be used as a Codex input.

It must not become a Codex permission grant.

13. Local checker and SYNAPS-style handoff

When the local checker is used near SYNAPS/Codex bridge flows, it must respect these defaults:

```
auto_ingest: false
memory: off
persistent: false
report_only: true
raw_evidence: no_content
secret_terms: forbidden
operator_pointer: minimal
```

A checker may refer to a quarantined transfer by id and hash.

It must not paste full contracts, payloads, tokens, patches, private logs, or raw evidence into ordinary chat or public reports.

14. Required finding classes

Class	Meaning	Example
LC-FILE	file inventory / canonical naming	duplicate (1) variant
LC-STATUS	stale status / overclaim	release notes say missing file that exists
LC-SCHEMA	schema structural problem	missing \$id
LC-SEM	semantic red-line violation	task attempts hack-back
LC-AB	AB/gate inconsistency	B used as authority
LC-REG	registry issue	revoked agent assigned task
LC-PERM	task/permission mismatch	grant exceeds contract
LC-SBX	sandbox/worktree issue	write target outside worktree
LC-WIT	witness issue	privileged transition lacks witness ref
LC-EVID	raw evidence / sidecar issue	raw evidence embedded in witness
LC-MEM	memory gate issue	agent output directly promoted
LC-ROLE	role separation issue	executor self-approves
LC-FIX	fixture safety issue	fixture contains live target
LC-RED	redaction issue	restricted detail in public surface
LC-REL	release surface issue	SHA generated before final artifact freeze
LC-CONF	conformance issue	conformance claimed without test run

15. Required blocker classes

The checker must emit BLOCK for:

1. any red-line authorization attempt;
2. any real secret in public or cloud-bound material;
3. any raw evidence in witness or memory;
4. any direct memory write by an agent;
5. any agent self-approval for material state change;
6. any revoked/suspended/quarantined agent assigned material task;
7. any missing task contract for material execution;
8. any permission grant exceeding task contract or registry ceiling;
9. any direct protected-branch write without release/deploy protocol;
10. any conformance claim without actual run evidence;
11. any public-safe claim without redaction pass;
12. any implementation-ready claim without schemas and validator path;
13. any release-complete claim with stale package-control status;

14. any unknown AB/apply semantics on material mutation;
15. any live external target in conformance fixture.

16. Safe report content

A local checker report may include:

- file paths inside package
- hashes
- schema ids
- finding ids
- status labels
- line/section references
- synthetic fixture ids
- sidecar ids
- redacted evidence refs
- witness refs
- allowed/blocked claims
- next required actions

A local checker report must not include:

- raw secrets
- raw private memory
- full prompt streams by default
- raw incident evidence
- legal privileged content
- real child/third-party sensitive data
- live target details
- provider account details
- private keys
- tokens
- canary values that weaken defense
- unredacted defensive garage contents

17. Implementation target layout

Recommended implementation layout:

```
tools/cgam_local_checker/  
  __init__.py  
  cli.py  
  inventory.py  
  status_hygiene.py  
  schema_check.py  
  semantic_guard.py  
  ab_gate_check.py  
  registry_check.py  
  task_permission_check.py  
  sandbox_check.py  
  witness_check.py  
  evidence_sidecar_check.py  
  memory_gate_check.py  
  role_separation_check.py  
  fixture_check.py  
  redaction_check.py  
  release_surface_check.py  
  conformance_check.py  
  report.py  
  policies/  
    local_checker_policy.v0.1.json  
  schemas/
```

```
cli-agent-local-checker-run.schema.json
cli-agent-local-checker-finding.schema.json
cli-agent-local-checker-result.schema.json
```

This layout is advisory for implementation.

The normative behavior is defined by this profile and parent documents.

18. Minimal CLI shape

Recommended command shape:

```
python -m tools.cgam_local_checker.cli \
  --mode LC-1 \
  --package-root ./docs/c-governed-cli-agent-mesh \
  --manifest PACKAGE_MANIFEST.json \
  --out reports/local_checker/run.json \
  --report-md reports/local_checker/run.md \
  --fail-on BLOCK
```

Required behavior:

```
exit 0 = PASS or PASS_WARN if --fail-on does not include WARN
exit 1 = FAIL
exit 2 = BLOCK
exit 3 = UNKNOWN under material gate
exit 4 = checker internal error
```

Checker internal error must not be interpreted as pass.

19. A/B behavior

19.1 Slot A

Slot A behavior:

```
read inputs
compute findings
write report only if report output is explicitly requested
no package mutation
no memory write
no quarantine clear
no release
no task execution
```

19.2 Slot B

Slot B behavior:

```
same checks as Slot A
may write checker report
may act as a blocking gate for another controlled process
may not itself perform the controlled process
```

Slot B still does not authorize mutation.

It only allows the checker result to be used as an enforceable gate by another layer.

20. Conformance tests for the checker

A checker implementation should pass at least these tests:

Test ID	Input	Expected result
LC-T001	missing canonical file	BLOCK for release-complete
LC-T002	duplicate sandbox file with (1) suffix	FAIL or BLOCK depending canonicity
LC-T003	release notes list existing file as missing	FAIL status hygiene
LC-T004	task contract without permission grant	BLOCK task-executable
LC-T005	permission grant exceeds task scope	BLOCK
LC-T006	revoked agent assigned material task	BLOCK
LC-T007	AB=B used without task contract	BLOCK
LC-T008	witness contains fake secret marker	FAIL witness/secret check
LC-T009	raw evidence embedded in memory proposal	BLOCK
LC-T010	synthetic fixture with live target URL	BLOCK
LC-T011	conformance-supported claim without run evidence	BLOCK
LC-T012	public package includes restricted incident profile unredacted	BLOCK
LC-T013	schema \$id missing	FAIL schema
LC-T014	LLM reviewer says “safe” but schema fails	FAIL; LLM text ignored
LC-T015	checker internal exception	non-pass exit

21. Relation to conformance

A local checker can support conformance.

It cannot by itself establish conformance.

Correct statements:

Local checker passed package hygiene preflight.
 Local checker passed schema structural preflight.
 Local checker found no red-line fixture violations.
 Local checker collected conformance run evidence references.

Incorrect statements:

The system is safe because the local checker passed.
 The package is conformant because schemas validate.
 The agent is trusted because registry validates.
 The release is public-safe because README exists.
 The incident is resolved because evidence packet exists.

22. Review and challenge

Every local checker run should be challengeable.

A review layer may ask:

Which policy version was used?
 Which files were included?
 Which files were excluded?
 Which findings were suppressed?
 Which warnings were accepted?
 Which unknowns were treated as not applicable?
 Which claim was being checked?
 Which evidence references support the result?

Suppression of findings must be explicit, witnessed where material, and never allowed for red-line blockers.

23. Open issues

ID	Issue	Status	Required action
LC-0I-001	Exact JSON schemas for checker run/finding/result not extracted	open	create schema files during schema extraction
LC-0I-002	Final package manifest format not fixed	open	align with release/public surface profile
LC-0I-003	Provider-specific checker rules absent	deferred	add after provider profiles
LC-0I-004	UI approval card not defined	open	align with future UI State Surface Profile
LC-0I-005	Exact severity thresholds for conformance scoring not fixed	open	add in conformance evidence packet profile
LC-0I-006	Integration with existing ester-clean-code validator/tooling not mapped file-by-file	open	create Codex implementation contract after implementation-ready gate

24. Release-readiness impact

This profile closes the conceptual gap identified as:

Local checker role is important but not detailed.
 Codex + semantic reviewer + local checker pattern needs local checker specifics.

After this file exists, the issue should be updated from:

OPEN: Local checker profile missing

to:

PARTIALLY_RESOLVED: profile exists; implementation, schemas, and runner still pending

This file does not make the package implementation-ready by itself.

It makes implementation-readiness definable.

25. Final invariant

A checker is a lock, not a king.
 A clean report is evidence, not sovereignty.
 A failed report is a stop signal.
 An unknown result is not permission.

The local checker exists so that worker agents cannot turn ambiguity into action.